

Проект по СПО

Паралелно пресмятане на числото на Ойлер (e) чрез конвейерна архитектура

Гаврил Христов

ФН: 6MI0800460

спец. Компютърни Науки
курс 3, втори поток, група 7

2026

Съдържание

1	Увод	2
1.1	Развитие на числото e в ред на Тейлър	2
1.2	Цел на проекта	2
2	Анализ на паралелизма	2
2.1	Методи за паралелно пресмятане на факториели и суми	2
2.2	Методи за балансиране на задачите	2
3	Проектиране и реализация	3
3.1	Функционално проектиране	3
3.2	Нефункционално проектиране	3
3.2.1	Характеристики на хардуерната платформа	3
3.2.2	Ръководство за потребителя	4
4	Тестване	4
4.1	Методология на тестване	4
4.2	Резултати	5
4.3	Анализ и обобщение	5
5	Източници	5

1 Увод

1.1 Развитие на числото e в ред на Тейлър

Числото e притежава уникалното свойство да бъде основа на естествената експоненциална функция. Математическата дефиниция чрез ред на Тейлър е:

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + \frac{1}{1!} + \frac{1}{2!} + \dots$$

Този ред осигурява бърза сходимост, тъй като факториелът в знаменателя нараства свръхекспоненциално. За постигане на точност от p десетични знака е необходимо да определим броя членове N така, че остатъкът $R_N \approx \frac{1}{(N+1)!}$ да бъде по-малък от 10^{-p} .

1.2 Цел на проекта

Основната цел на проекта е проектирането и реализацията на паралелна система за изчисляване на числото e с висока точност. Системата използва конвейерна архитектура за разпределяне на изчисленията, с цел постигане на оптимална скалируемост чрез балансирано натоварване на нишките спрямо сложността на изчисление на факториелите.

2 Анализ на паралелизма

2.1 Методи за паралелно пресмятане на факториели и суми

Предизвикателството при паралелното пресмятане на редове от типа на Тейлър се състои в последователната зависимост на факториелите. В проекта използваме декомпозиция на домейна, при която изчислителният обхват $[0, N]$ се разделя на интервали. Паралелизмът се реализира чрез конвейерна обработка, при която всяка нишка предава междинния си резултат на следващата, след като е завършила пресмятането на своя интервал. Този подход минимизира конфликтите при достъп до паметта [2].

2.2 Методи за балансиране на задачите

Апроксимацията на Стирлинг е фундаментална за нашия проект, тъй като позволява предварително определяне на броя членове N в реда на Тейлър, необходими за постигане на желаната точност p . Прилагайки логаритмична трансформация, изискването е $\ln((N+1)!) > p \cdot \ln(10)$. Използвайки апроксимацията на Стирлинг $\ln(n!) \approx n \ln n - n$, неравенството се трансформира до $(N+1) \ln(N+1) - (N+1) \approx p \ln(10)$. При големи стойности на N , пренебрегвайки по-ниските степени, получаваме $N \ln N \approx p \ln 10$. Премахвайки към десетичен логаритъм, достигаме до практическата формула $N \approx \frac{p}{\log_{10} p}$, която използваме за планиране на работния диапазон.

Тъй като изчисляването на факториел с голяма стойност е изчислително скъпо, ние използваме тази форма, за да установим границата, при която остатъкът R_N става по-малък от 10^{-p} . Методът за определяне на големината на интервалите е реализиран чрез функцията `distribute_work` в `helper.py`. Изчислителният обхват $[0, N]$ се разделя на $T \times I$ интервала, където T е броят на нишките, а I е коефициентът на грануларност. Всеки интервал се изчислява чрез линейно разпределение: $[\frac{N \cdot i}{C}, \frac{N \cdot (i+1)}{C}]$, където C е общият брой интервали. Този подход осигурява равномерно натоварване на работните нишки, като минимизира времето за престой на отделните процесори в конвейерната архитектура.

3 Проектиране и реализация

3.1 Функционално проектиране

Системата е изградена върху линейен (конвейерен) модел на паралелизъм, който позволява разпределено изчисляване на сумата на реда на Тейлър. Архитектурата е организирана в логическа последователност от етапи, които осигуряват поточен пренос на данни и ефективно натоварване на процесорните ресурси. Функционалният модел включва следните компоненти:

- **Подготовка и декомпозиция на домейна:** Първоначалният етап включва оценка на математическата сложност на задачата — определяне на броя членове от реда, необходими за постигане на зададената точност. Изчислителното пространство се разделя на под-интервали, като разпределението на натоварването се адаптира спрямо експоненциалния характер на факториелите в знаменателя. Това осигурява балансираност на работата между отделните нишки.
- **Конвейерна синхронизация:** Преносът на информация между различните нишки се осъществява чрез механизъм за опашки, който осигурява надеждна връзка между етапите на конвейера. Всеки етап получава междинен резултат от предходния, актуализира го с локално изчислената частична сума и предава новата стойност към следващия сегмент.
- **Изчислителен процес:** Логиката на изчисление е декомпозирана така, че всеки работен процес извършва локално сумиране на своя специфичен сегмент. Този подход минимизира конфликтите при достъп до споделени ресурси и позволява едновременното пресмятане на частични суми от различни части на реда.
- **Управление на точността:** Поради специфичните изисквания за висока прецизност при пресмятане на ирационални числа, системата използва аритметика с произволна точност. Контекстът на изчисленията се дефинира динамично, за да се предотврати натрупването на грешки от закръгляне и да се гарантира коректността на крайния резултат.

Координацията на целия процес е централизирана, като се контролира жизненият цикъл на нишките и се осигурява коректно обединяване на междинните резултати от финалната фаза на конвейера.

3.2 Нефункционално проектиране

3.2.1 Характеристики на хардуерната платформа

За нуждите на разработката, тестването и анализа на скалируемостта е използвана следната конфигурация:

Категория	Параметър	Стойност
CPU	Модел	Intel Core i7-13700K
	Архитектура	x86_64
	Общо ядра	16 (8 P-ядра, 8 E-ядра)
	Нишки	24
	Честота	3.4 GHz (Base) / 5.4 GHz (Turbo)
	Кеш (L1/L2/L3)	1.4 MB / 24 MB / 30 MB
RAM	Размер	64 GB
OS	Версия	Windows 11

3.2.2 Ръководство за потребителя

Системата изисква инсталирана версия на Python 3.14 (Free-Threading/NoGIL), която е необходима за постигане на истинско паралелно изпълнение без ограниченията на глобалния интерпретаторски ключ (GIL).

Използвани пакети и модули: Проектът използва вградените модули на стандартната библиотека:

- decimal – за прецизна аритметика с произволна точност.
- threading – за управление на жизнения цикъл на нишките.
- queue – за реализация на поточната синхронизация.
- argparse – за парсане на аргументите от командния ред.
- math, os, time, sys – за математически функции и системни операции.

Изпълнение на програмата: Програмата се стартира през терминала чрез основния скрипт main.py. Синтаксисът е следният:

```
python main.py [-t THREADS] -p PRECISION [-i INTERVAL] [-q] file
```

Описание на аргументите:

- -t, --threads (опционален): Брой нишки, които ще се използват за пресмятането. По подразбиране е 1.
- -p, --precision (задължителен): Брой десетични знаци, до които да се пресметне числото e .
- -i, --interval (опционален): Коефициент на грануларност (брой интервали на нишка). По подразбиране е 1.
- -q, --quiet (флаг): Потиска изхода в конзолата (използва се при автоматизирани процеси).
- file (позиционен аргумент): Пътят до файла, в който ще бъде записан крайният резултат.

4 Тестване

4.1 Методология на тестване

За оценка на производителността и скалируемостта на реализираната система е разработен тестов план, базиран на следните ключови параметри, използвани за анализ на паралелното изпълнение:

- p (**точност**): Брой десетични знаци, до които се пресмята числото e . Този параметър директно определя сложността на изчислението и броя на членовете от реда на Тейлър.
- g (**грануларност**): Коефициент на сегментиране (интервали на нишка), който дефинира фиността на разпределението на изчислителните задачи.
- $T_p(i)$: Измерено време за изпълнение на конкретен опит (тест) с номер i при използване на зададен брой нишки p .

- $T_p = \min(T_p(i))$: Най-доброто (минимално) постигнато време измежду всички проведени опити за конкретна конфигурация. Тази стойност служи като база за изчисляване на показателите за производителност.
- $S_p = T_1/T_p$: Ускорение, показващо съотношението между времето за последователно изпълнение (T_1) и най-доброто време за паралелно изпълнение с p нишки (T_p).
- $E_p = S_p/p$: Ефективност, представяща колко ефективно се използват наетите ресурси (нишки) в сравнение с идеалното ускорение.

4.2 Резултати

[Тук следва да бъдат поставени таблиците с данни от тестовете и съответните графики, получени при изпълнението на бенчмарк програмата.]

4.3 Анализ и обобщение

[Тук следва да бъде направен анализ на получените резултати, коментар относно скалируемостта на програмата и нейните ограничения при различна натовареност.]

5 Източници

- [1] Haible, B., & Papanikolaou, T. (1997). *Fast multiprecision evaluation of series of rational numbers*. Proceedings of the 3rd International Symposium on Symbolic and Algebraic Computation. <https://dl.acm.org/doi/10.1145/258726.258814>
- [2] Kamruzzaman, M., Swanson, S., & Tullsen, D. M. (2013). *Load-balanced pipeline parallelism*. Proceedings of the International Conference on High Performance Computing (SC'13). <https://scispace.com/pdf/load-balanced-pipeline-parallelism-3ad4huixe2.pdf>
- [3] Vestermarck, J. (2025). *Optimization of Binary Splitting for Series Evaluation*. Computational Mathematics Journal. <https://www.researchgate.net/publication/optimization-binary-splitting>
- [4] Brent, R. P. (1976). *Fast Multiple-Precision Evaluation of Elementary Functions*. Journal of the ACM (JACM), 23(2), 242-251. <https://dl.acm.org/doi/10.1145/321941.321944>